What Fine-Tuning Actually Costs: A Measured Comparison of Full, LoRA and QLoRA Adaptation

Chandra Prakash Bathula
Saint Louis University
chandraprakash.bathula@slu.edu

Ram Mohan Bathula
Indira Gandhi National Open University
rammohan.bathula9@gmail.com

Abstract

LoRA and QLoRA are often presented mainly as ways to reduce the memory needed for fine-tuning. We wanted to measure what that trade-off looks like in practice. We ran eleven fine-tunes of the same 7.62B model on the same task: full fine-tuning, plus LoRA and QLoRA at five ranks each. We evaluated the models by executing generated SQL against the database rather than comparing SQL strings directly. Execution accuracy ranged from 75.4% to 78.3%, while checkpoint size differed by up to 707× and trainable parameters by 3,018×. Because all variants answer the same 1,000 questions, we compare them with paired tests (McNemar, paired bootstrap, paired permutation, Cochran's Q). Across 55 pairwise comparisons, none survives Holm correction, and no rank difference is detectable (Cochran's Q, $p = 0.46$ across LoRA ranks, $p = 0.14$ across QLoRA ranks). The ten adapters average 2.0 points above full fine-tuning (95% CI 0.4–3.6, $p = 0.013$), a difference confounded with learning rate and based on one seed. All reported numbers come from experiment artifacts or arithmetic on them.

1 The Concept

Parameter-efficient fine-tuning (PEFT) teaches how large language models can be adapted to new tasks without updating every model parameter. This is important because full fine-tuning can require substantial GPU memory and produce large checkpoints, making adaptation expensive even when the underlying task is relatively small. LoRA addresses this by freezing the base model and learning a low-rank update $\Delta W = BA$, while QLoRA combines the same adaptation with a 4-bit NF4-quantized base model. The material makes these trade-offs concrete through eleven fine-tunes of Qwen2.5-7B-Instruct on WikiSQL, comparing full fine-tuning with LoRA and QLoRA across ranks 4, 8, 16, 32, and 64. In our experiments, execution accuracy ranged from 75.4% to 78.3% and the adapters outperformed full fine-tuning by 2.0 points on average, while checkpoint sizes differed by up to 707×, illustrating that substantial resource savings need not cost task performance. The material also highlights a related evaluation issue: SQL generation should be evaluated by execution rather than exact string matching because different SQL queries can produce identical results. These topics remain central to current LLM research as practitioners seek efficient adaptation methods for increasingly large models under limited compute and memory budgets.

Scope. These results are for one model (Qwen2.5-7B-Instruct), one task (WikiSQL), one seed per variant, and one learning rate per method (10^{-5} for full fine-tuning, 2×10^{-4} for adapters). Accuracies are taken from re-generated per-question outputs, which the paired tests require. Re-generating changes individual accuracies by up to 0.4 points relative to the original evaluation, which sets a practical noise floor.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Education Track.

2 Leveling and Prerequisite Knowledge

The material is intended for **advanced undergraduate and graduate** learners and is designed to fit a 90-minute laboratory or a unit within a graduate seminar. Learners should be comfortable with basic neural-network training, matrix multiplication, matrix rank, optimizer states, and reading Python/PyTorch code. These prerequisites are sufficient to follow the main comparisons and understand the parameter and memory calculations. No prior knowledge of LoRA, QLoRA, low-bit quantization, distributed training, or parameter-efficient fine-tuning is required.

The material supports different levels of engagement. Learners with limited fine-tuning experience can focus on the interactive comparisons of model outputs, accuracy, memory, and checkpoint size, while more advanced learners can examine the low-rank parameterization, quantization, memory calculations, and statistical comparisons. Required concepts are introduced immediately before they are used, making the resource suitable for both a focused advanced-undergraduate laboratory and graduate-level teaching.

3 Learning Objectives and Outcomes

A learner who completes the material will be able to:

- **Remember** what LoRA and QLoRA freeze and how the low-rank update $\Delta W = BA$ is used.
- **Understand** why freezing parameters reduces training memory and why 4-bit storage does not imply 4-bit computation.
- **Apply** $r(d+k)$ versus dk to estimate trainable parameters before training.
- **Analyse** accuracy, peak memory, checkpoint size, and training time across fine-tuning methods.
- **Evaluate** accuracy differences using paired tests (McNemar, paired bootstrap, paired permutation, Cochran's Q), with a multiple-comparison correction when many pairs are examined.
- **Create** a fine-tuning plan for a given compute or storage budget, including a justified rank and base precision.

Outcome. Learners use measured results to make informed fine-tuning choices rather than assuming that lower-cost methods are less accurate. They also learn to account for deployment conditions when selecting an adaptation method, including the effect of changing the base model precision after training. The eleven-run comparison provides concrete evidence for reasoning about these trade-offs, including where QLoRA provides additional memory savings over LoRA. It also shows learners that the same accuracy gap can look non-significant under an unpaired test and significant under a paired one, and that picking the widest pair after the fact inflates apparent significance unless the test is corrected for the number of comparisons.

4 Linked Papers

The low-rank re-parameterization and merge-at-inference property are introduced in [3], while [2] introduces the NF4 quantization, double quantization, and paged optimizers used by QLoRA. [1] shows that the gap between LoRA and full fine-tuning depends on the training regime, with larger differences during continued pretraining and smaller differences under instruction fine-tuning at higher ranks. [5] further shows that similar task scores do not imply identical learned solutions. [4] demonstrates the serving benefits of compact LoRA adapters, while [6] examines parameter-efficient fine-tuning across 310 models. We use WikiSQL [7] so that generated SQL can be evaluated directly against the database without relying on an LLM judge.

5 Teaching Materials Summary

The submission provides an **interactive fine-tuning laboratory**, a **reproducible experiment replay**, and supporting code files and artifacts built from eleven real fine-tuning experiments on

Qwen/Qwen2.5-7B-Instruct over WikiSQL. The four-part lesson covers full fine-tuning, LoRA, QLoRA, and a comparison playground. Learners can explore model outputs, accuracy, memory, checkpoint size, training curves, and rank-dependent trade-offs using recorded experimental results.

The LoRA lesson includes an eight-step walkthrough of $\Delta W = BA$ using a real `q_proj`, while the QLoRA lesson provides an interactive NF4 quantization example and memory calculations. The playground allows learners to replay experiments, compare variants across metrics, and inspect all 1,000 held-out questions. On 48 of them all eleven variants are right despite writing different SQL, and 109 are missed by all eleven (54 of those with differing SQL).

Every displayed result traces to a recorded artifact, model configuration, or explicit calculation. The eleven experiment artifacts, registry, per-question outputs, paired significance tests (`scripts/paired_stats.py`), figures, analysis scripts, and application source code are included so that the results can be inspected and reproduced without retraining the models. An optional inference backend is also provided for live experimentation on suitable CUDA hardware.

References

[1] Biderman, D., Portes, J., Ortiz, J. J. G., Paul, M., Greengard, P., Jennings, C., King, D., Havens, S., Chiley, V., Frankle, J., Blakeney, C., & Cunningham, J. P. (2024). LoRA learns less and forgets less. https://arxiv.org/abs/2405.09673

[2] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. https://arxiv.org/abs/2305.14314

[3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. International Conference on Learning Representations (ICLR). https://openreview.net/forum?id=nZeVKeeFYf9

[4] Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., Gonzalez, J. E., & Stoica, I. (2024). S-LoRA: Serving thousands of concurrent LoRA Adapters. https://arxiv.org/abs/2311.03285

[5] Shuttleworth, R., Andreas, J., Torralba, A., & Sharma, P. (2025). LoRA vs full fine-tuning: An illusion of equivalence. https://arxiv.org/abs/2410.21228

[6] Zhao, J., Wang, T., Abid, W., Angus, G., Garg, A., Kinnison, J., Sherstinsky, A., Molino, P., Addair, T., & Rishi, D. (2024). LoRA Land: 310 Fine-tuned LLMs that rival GPT-4, a technical report. https://arxiv.org/abs/2405.00732

[7] Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating structured queries from natural language using reinforcement learning. https://arxiv.org/abs/1709.00103

6 Resources

GitHub: `https://github.com/ChandraPrakash-Bathula/FineTuning`
Hugging Face Space: `https://huggingface.co/spaces/ChandraPrakashBathula/finetuning-laboratory`